Lab 4: VibeCoding - Chat History (exported transcript)

Aryan Upadhyay | SRN: PES1UG25CS806 | Section C | Repo: SETAPESU26/59_rock_paper_scissor | Tool: Claude Code (VS Code), model Claude Opus 5.5
Exported from the Claude Code session log. Includes every message and tool call (commands, file writes) for Lab 4; long tool outputs are shortened, screenshots shown as [image], local paths abbreviated, and the parts of the same session about Labs 1-3 left out.

Student · 2026-10-02 17:14 UTC

complete all the labs and upload on my github

Claude · 2026-10-02 17:14 UTC

I'll start by looking at what's in the lab folder.

Tool call - Bash: List lab folder contents

```
cd "/Users/aryan.upadhyayyy/Desktop/SE Lab " && ls -la && find . -maxdepth 3 -not -path '*/.*' | head -100
```

Result

```
total 10112
drwxr-xr-x@  9 aryan.upadhyayyy  staff      288 Sep 30 12:13 .
drwx------+ 23 aryan.upadhyayyy  staff      736 Oct  1 13:44 ..
-rw-r--r--@  1 aryan.upadhyayyy  staff     6148 Sep 30 12:35 .DS_Store
-rw-------@  1 aryan.upadhyayyy  staff   154016 Sep 30 12:04 Lab1_RE_student_handout.pdf
-rw-------@  1 aryan.upadhyayyy  staff  4347655 Sep 30 12:06 Lab_2_Jira_Student_Handout.pdf
-rw-------@  1 aryan.upadhyayyy  staff   214435 Sep 30 12:07 Lab_3_Architecture.pdf
-rw-------@  1 aryan.upadhyayyy  staff    71355 Sep 30 12:12 Lab_4_VibeCoding_handout.pdf
-rw-------@  1 aryan.upadhyayyy  staff   267604 Sep 30 12:05 Problem statement-SRN-C-Section.pdf
-rw-------@  1 aryan.upadhyayyy  staff    98509 Sep 30 12:12 VibeCodingGitRepoLinks-Sec-C.pdf
.
./Lab1_RE_student_handout.pdf
./Lab_2_Jira_Student_Handout.pdf
./Lab_4_VibeCoding_handout.pdf
./VibeCodingGitRepoLinks-Sec-C.pdf
./Lab_3_Architecture.pdf
./Problem statement-SRN-C-Section.pdf
```

Claude · 2026-10-02 17:14 UTC

Six PDFs. I'll read them all to understand what each lab asks for.

Tool call - Bash: Check available PDF tools

```
cd "/Users/aryan.upadhyayyy/Desktop/SE Lab " && which pdftotext; python3 -c "import pypdf; print('pypdf ok')" 2>&1;
  python3 -c "import fitz; print('fitz ok')" 2>&1
```

Result

```
Exit code 1
pdftotext not found
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'pypdf'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'fitz'
```

Tool call - Read <SE Lab>/Lab1_RE_student_handout.pdf

```
{
 "file_path": "<SE Lab>/Lab1_RE_student_handout.pdf"
}
```

Result

```
PDF file read: <SE Lab>/Lab1_RE_student_handout.pdf (150.4KB)
[image]
```

Tool call - Read <SE Lab>/Problem statement-SRN-C-Section.pdf

```
{
 "file_path": "<SE Lab>/Problem statement-SRN-C-Section.pdf"
}
```

Result

```
PDF file read: <SE Lab>/Problem statement-SRN-C-Section.pdf (261.3KB)
[image]
```

Tool call - Read <SE Lab>/VibeCodingGitRepoLinks-Sec-C.pdf

```
{
 "file_path": "<SE Lab>/VibeCodingGitRepoLinks-Sec-C.pdf"
}
```

Result

```
PDF file read: <SE Lab>/VibeCodingGitRepoLinks-Sec-C.pdf (96.2KB)
[image]
```

Tool call - Read <SE Lab>/Lab_3_Architecture.pdf

```
{
 "file_path": "<SE Lab>/Lab_3_Architecture.pdf"
}
```

Result

```
PDF file read: <SE Lab>/Lab_3_Architecture.pdf (209.4KB)
[image]
```

Tool call - Read <SE Lab>/Lab_4_VibeCoding_handout.pdf

```
{
 "file_path": "<SE Lab>/Lab_4_VibeCoding_handout.pdf"
}
```

Result

```
PDF file read: <SE Lab>/Lab_4_VibeCoding_handout.pdf (69.7KB)
[image]
```

Tool call - Read <SE Lab>/Lab_2_Jira_Student_Handout.pdf

```
{
 "file_path": "<SE Lab>/Lab_2_Jira_Student_Handout.pdf"
}
```

Result

```
PDF file read: <SE Lab>/Lab_2_Jira_Student_Handout.pdf (4.1MB)
[image]
```

Tool call - Bash: Check git, GitHub CLI auth, and Python setup

```
which gh git python3; gh auth status 2>&1; git config --global user.name; git config --global user.email; python3 --
  version; python3 -c "import pygame; print('pygame', pygame.version.ver)" 2>&1 | tail -1
```

Tool call - Bash: Clone the assigned Lab 4 repo into scratchpad

```
cd "/private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad"
  && git clone https://github.com/SETAPESU26/59_rock_paper_scissor.git 2>&1 && cd 59_rock_paper_scissor && git log --
  oneline | head && find . -type f -not -path './.git/*' | head -50
```

Result

```
gh not found
/usr/bin/git
/usr/bin/python3
(eval):1: command not found: gh
Python 3.9.6
ModuleNotFoundError: No module named 'pygame'
```

Result

```
Cloning into '59_rock_paper_scissor'...
72e89d3 Update game_engine.py
60c0604 Update README.md
86607ad Add files via upload
./README.md
./main.py
./game/game_engine.py
./game/button.py
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:14 UTC

Understood: you want every deliverable produced and pushed. I'll read the Lab 4 code first.

Tool call - Bash: Print all files of the Lab 4 repo

```
cd "<scratch>/59_rock_paper_scissor" && for f in README.md main.py game/game_engine.py game/button.py; do echo "=====
  $f"; cat -n "$f"; done
```

Result

```
===== README.md
     1■# Rock Paper Scissor Lab
     2■
     3■This project is an interactive hand-game duel using **Pygame**. It introduces students to dictionary-based
  rule lookups, automated CPU decision-making, temporary result display timers, and modular UI button interaction
  within an object-oriented codebase.
     4■---
     5■
     6■## What's Provided
     7■
     8■A working Rock Paper Scissors game with:
     9■
    10■- Three interactive choice buttons (`ROCK`, `PAPER`, `SCISSORS`) with mouse hover and click response states
    11■- An automated computer opponent selecting moves at random
    12■- Live scoreboard tracking player score and CPU score independently
    13■- Timed result display that shows the outcome for 1.8 seconds before clearing picks for the next round
    14■
    15■It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to
  **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more
  interesting.
    16■
    17■### **Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.**
    18■---
    19■
    20■## Getti
... [10587 more characters omitted]
```

Tool call - AskUserQuestion

```
{
 "questions": [
  {
   "question": "Your assigned Problem Statement is #60, but the PDF doesn't contain the scenario text. What should
  Labs 1\u20133 be based on?",
   "header": "Scenario",
   "multiSelect": false,
   "options": [
    {
     "label": "I'll paste #60 text",
     "description": "Choose 'Other' and paste the problem statement #60 scenario; I'll build Labs 1\u20133 on it."
```

```
      },
      {
       "label": "Use Coffee Kiosk",
       "description": "Use the Self-Service Coffee Kiosk example that the Lab 1\u20133 handouts are written around."
      }
     ]
    },
    {
     "question": "How should I push to your GitHub? (gh CLI isn't installed and git has no user configured.)",
     "header": "GitHub",
     "multiSelect": false,
     "options": [
      {
       "label": "Install gh, I'll log in (Recommended)",
       "description": "I install gh via Homebrew; you run `! gh auth login` in this prompt; then I create a repo (e.g.
   SE-Lab) and push Lab-1\u2026Lab-4 folders."
      },
      {
       "label": "I'll give a repo URL",
       "description": "You create an empty repo on github.com and paste its URL plus your GitHub username via 'Other';
   I push over HTTPS using your saved credentials."
      }
     ]
    }
   ]
  }
```

Result

```
Your questions have been answered: "Your assigned Problem Statement is #60, but the PDF doesn't contain the scenario
  text. What should Labs 1–3 be based on?"="Use Coffee Kiosk", "How should I push to your GitHub? (gh CLI isn't
  installed and git has no user configured.)"="Install gh, I'll log in (Recommended)". You can now continue with
  these answers in mind.
```

Tool call - Bash: Check Homebrew and ffmpeg availability

```
which brew ffmpeg; brew --version 2>&1 | head -1
```

Result

```
brew not found
ffmpeg not found
(eval):1: command not found: brew
```

Claude · 2026-10-02 17:16 UTC

Homebrew isn't installed, so I'll download the official gh binary from GitHub's releases page instead.

Tool call - Bash: Download GitHub CLI binary into ~/.local/bin

```
uname -m; V=$(curl -s https://api.github.com/repos/cli/cli/releases/latest | python3 -c "import sys,json;print(json.
  load(sys.stdin)['tag_name'].lstrip('v'))"); echo $V; ARCH=$( [ "$(uname -m)" = arm64 ] && echo arm64 || echo amd64
  ); cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/
  scratchpad && curl -sL -o gh.zip "https://github.com/cli/cli/releases/download/v${V}/gh_${V}_macOS_${ARCH}.zip" &&
  unzip -oq gh.zip && mkdir -p ~/.local/bin && cp gh_${V}_macOS_${ARCH}/bin/gh ~/.local/bin/gh && ~/.local/bin/gh --
  version
```

Result

```
arm64
2.102.0
gh version 2.102.0 (2026-09-30)
https://github.com/cli/cli/releases/tag/v2.102.0
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Bash: Create venv and install pygame, docx, reportlab, ffmpeg

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  python3 -m venv venv && ./venv/bin/pip install -q --upgrade pip && ./venv/bin/pip install -q pygame python-docx
  reportlab imageio imageio-ffmpeg numpy 2>&1 | tail -3; ./venv/bin/python -c "import pygame, docx, reportlab,
  imageio_ffmpeg; print('ok', pygame.version.ver, imageio_ffmpeg.get_ffmpeg_exe())"
```

Result

```
pygame 2.6.1 (SDL 2.28.4, Python 3.9.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
ok 2.6.1 <scratch>/venv/lib/python3.9/site-packages/imageio_ffmpeg/binaries/ffmpeg-macos-aarch64-v7.1
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Bash: Keep pristine copy of original game; install PDF renderer

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  cp -R 59_rock_paper_scissor rps_original && ./venv/bin/pip install -q pymupdf 2>&1 | tail -1; ./venv/bin/python -c
  "import fitz; print('fitz ok')"
```

Result

```
fitz ok
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:18 UTC

Now the Lab 4 code. I'll fix the bug, then add the three README features: first-to-X match win, adaptive AI, and drawn gesture icons with a reveal animation.

Tool call - Write <scratch>/59_rock_paper_scissor/game/icons.py

```
import math
import pygame


def draw_rock(surface, center, size):
    cx, cy = center
    r = size // 2
```

```
    # Lumpy boulder outline built from a perturbed circle
    points = []
    bumps = [1.0, 0.92, 1.04, 0.95, 1.02, 0.9, 1.05, 0.94, 1.0, 0.93, 1.03, 0.96]
    for i, k in enumerate(bumps):
        angle = 2 * math.pi * i / len(bumps)
        points.append((cx + math.cos(angle) * r * k, cy + math.sin(angle) * r * k * 0.85))
    pygame.draw.polygon(surface, (128, 128, 140), points)
    pygame.draw.polygon(surface, (70, 70, 82), points, width=max(2, size // 30))

    # Knuckle creases and a highlight to read as a clenched fist / stone
    crease = (90, 90, 102)
    for i in range(3):
        x = cx - r * 0.45 + i * r * 0.45
        pygame.draw.arc(surface, crease, pygame.Rect(x - r * 0.2, cy - r * 0.55, r * 0.4, r * 0.4), math.pi, 2 * math.
  pi, max(2, size // 40))
    pygame.draw.circle(surface, (170, 170, 182), (int(cx - r * 0.35), int(cy + r * 0.15)), max(2, size // 12))


def draw_paper(surface, center, size):
    cx, cy = center
    w, h = int(size * 0.72), int(size * 0.92)
    rect = pygame.Rect(cx - w // 2, cy - h // 2, w, h)
    fold = size // 5

    sheet = [
        rect.topleft,
        (rect.right - fold, rect.top),
        (rect.right, rect.top + fold),
        rect.bottomright,
        rect.bottomleft,
    ]
    pygame.draw.polygon(surface, (240, 242, 248), sheet)
    pygame.draw.polygon(surface, (120, 130, 150), sheet, width=max(2, size // 30))

    # Dog-eared corner
    corner = [(rect.right - fold, rect.top), (rect.right - fold, rect.top + fold), (rect.right, rect.top + fold)]
    pygame.draw.polygon(surface, (200, 205, 218), corner)
    pygame.draw.polygon(surface, (120, 130, 150), corner, width=max(1, size // 40))

    # Ruled lines
    line_gap = h // 7
    for i in range(2, 7):
        y = rect.top + i * line_gap
        pygame.draw.line(surface, (150, 175, 215), (rect.left + w // 8, y), (rect.right - w // 8, y), max(1, size //
  45))


def draw_scissors(surface, center, size):
    cx, cy = center
    r = size // 2
    pivot = (cx, cy + r * 0.05)
    blade = (205, 210, 220)
    edge = (90, 95, 110)

    # Two blades crossing in an X above the pivot
    for side in (-1, 1):
        tip = (cx + side * r * 0.55, cy - r * 0.95)
        base_l = (pivot[0] - side * r * 0.12, pivot[1] + r * 0.05)
        base_r = (pivot[0] + side * r * 0.02, pivot[1] - r * 0.12)
        poly = [base_l, tip, base_r]
        pygame.draw.polygon(surface, blade, poly)
        pygame.draw.polygon(surface, edge, poly, width=max(1, size // 40))

    # Finger rings below the pivot
    ring_r = int(r * 0.3)
    for side in (-1, 1):
        ring_c = (int(cx + side * r * 0.4), int(cy + r * 0.6))
        pygame.draw.line(surface, (200, 60, 60), pivot, ring_c, max(3, size // 14))
        pygame.draw.circle(surface, (200, 60, 60), ring_c, ring_r, width=max(3, size // 14))

    pygame.draw.circle(surface, (60, 60, 70), (int(pivot[0]), int(pivot[1])), max(3, size // 16))


_DRAWERS = {"ROCK": draw_rock, "PAPER": draw_paper, "SCISSORS": draw_scissors}


def draw_gesture(surface, choice, center, size):
    drawer = _DRAWERS.get(choice)
    if drawer:
        drawer(surface, center, size)
```

Result

```
File created successfully at: <scratch>/59_rock_paper_scissor/game/icons.py (file state is current in your context —
  no need to Read it back)
```

Tool call - Write <scratch>/59_rock_paper_scissor/game/button.py

```
import pygame
from game.icons import draw_gesture


class ChoiceButton:
    def __init__(self, choice_name, rect, color, hover_color):
        self.choice_name = choice_name
        self.rect = rect
```

```
        self.color = color
        self.hover_color = hover_color
        self.font = pygame.font.SysFont(None, 26)

    def contains(self, pos):
        return self.rect.collidepoint(pos)

    def render(self, surface, enabled=True):
        mouse_pos = pygame.mouse.get_pos()
        hovered = enabled and self.rect.collidepoint(mouse_pos)
        bg_color = self.hover_color if hovered else self.color
        if not enabled:
            bg_color = tuple(c // 2 for c in self.color)

        pygame.draw.rect(surface, bg_color, self.rect, border_radius=8)
        pygame.draw.rect(surface, (230, 230, 235), self.rect, width=2, border_radius=8)

        # Small gesture icon on the left, label on the right
        icon_size = self.rect.height - 16
        icon_center = (self.rect.left + 12 + icon_size // 2, self.rect.centery)
        draw_gesture(surface, self.choice_name, icon_center, icon_size)

        text_surf = self.font.render(self.choice_name, True, (255, 255, 255))
        text_left = self.rect.left + 20 + icon_size
        text_area = self.rect.right - text_left
        surface.blit(
            text_surf,
            (
                text_left + (text_area - text_surf.get_width()) // 2,
                self.rect.centery - text_surf.get_height() // 2,
            ),
        )
```

Result

```
The file <scratch>/59_rock_paper_scissor/game/button.py has been updated successfully. (file state is current in your
  context — no need to Read it back)
```

Tool call - Write <scratch>/59_rock_paper_scissor/game/game_engine.py

```
import math
import random
from collections import Counter, deque

import pygame
from game.button import ChoiceButton
from game.icons import draw_gesture

# Which move beats which: key beats value
BEATS = {"ROCK": "SCISSORS", "SCISSORS": "PAPER", "PAPER": "ROCK"}
# The move that beats the key
COUNTER = {loser: winner for winner, loser in BEATS.items()}

STATE_IDLE = "IDLE"
STATE_COUNTDOWN = "COUNTDOWN"
STATE_RESULT = "RESULT"
STATE_MATCH_OVER = "MATCH_OVER"


class GameEngine:
    def __init__(self, width, height, target_wins=5):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 160, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 70

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50, 50), (200, 70, 70)),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (40, 100, 170), (60, 130,
  210)),
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (180, 140, 30), (
  220, 180, 50)),
        ]

        # Match settings
        self.target_wins = target_wins

        # Adaptive AI settings: look at the player's last N throws
        self.history_size = 6
        self.bias_strength = 1.5

        self.display_duration = 1800
        self.countdown_beat = 350
        self.countdown_words = ["ROCK...", "PAPER...", "SCISSORS..."]
        self.countdown_duration = self.countdown_beat * len(self.countdown_words)
        self.reveal_pop = 220

        self.font_title = pygame.font.SysFont(None, 36)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_arena = pygame.font.SysFont(None, 32)
        self.font_small = pygame.font.SysFont(None, 22)
```

```
    self.font_banner = pygame.font.SysFont(None, 54)

    self.reset_match()

def reset_match(self):
    self.player_choice = None
    self.cpu_choice = None
    self.result_text = "Make your move!"
    self.result_color = (220, 225, 235)

    self.player_score = 0
    self.cpu_score = 0
    self.match_winner = None

    self.player_history = deque(maxlen=self.history_size)
    self.ai_note = ""

    self.state = STATE_IDLE
    self.state_started = pygame.time.get_ticks()

def determine_winner(self, player, cpu):
    if player == cpu:
        return "TIE"
    return "PLAYER" if BEATS[player] == cpu else "CPU"

def cpu_pick(self):
    """Pick a CPU move, biased toward countering the player's recent favourites."""
    weights = {choice: 1.0 for choice in self.choices}
    counts = Counter(self.player_history)
    for move, count in counts.items():
        weights[COUNTER[move]] += count * self.bias_strength

    if counts:
        favourite, fav_count = counts.most_common(1)[0]
        if fav_count >= 3:
            self.ai_note = f"CPU has noticed you favour {favourite}..."
        else:
            self.ai_note = ""

    return random.choices(self.choices, weights=[weights[c] for c in self.choices])[0]

def play_round(self, choice):
    self.player_choice = choice
    # The CPU decides from history *before* this throw so it can't peek
    self.cpu_choice = self.cpu_pick()
    self.player_history.append(choice)

    self.state = STATE_COUNTDOWN
    self.state_started = pygame.time.get_ticks()

def resolve_round(self):
    outcome = self.determine_winner(self.player_choice, self.cpu_choice)
    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}."
        self.result_color = (80, 230, 120)
    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}."
        self.result_color = (240, 80, 80)
    else:
        self.result_text = f"It's a Draw! Both picked {self.player_choice}."
        self.result_color = (240, 210, 80)

    if self.player_score >= self.target_wins:
        self.match_winner = "PLAYER"
    elif self.cpu_score >= self.target_wins:
        self.match_winner = "CPU"

    self.state = STATE_RESULT
    self.state_started = pygame.time.get_ticks()

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        if self.state == STATE_MATCH_OVER:
            self.reset_match()
        return

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.state not in (STATE_IDLE, STATE_RESULT):
            return
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break

def update(self):
    elapsed = pygame.time.get_ticks() - self.state_started

    if self.state == STATE_COUNTDOWN and elapsed >= self.countdown_duration:
        self.resolve_round()
    elif self.state == STATE_RESULT and self.match_winner and elapsed >= self.display_duration:
        self.state = STATE_MATCH_OVER
        self.state_started = pygame.time.get_ticks()
```

```
        elif self.state == STATE_RESULT and elapsed >= self.display_duration:
            self.player_choice = None
            self.cpu_choice = None
            self.result_text = "Make your move!"
            self.result_color = (190, 195, 205)
            self.state = STATE_IDLE
            self.state_started = pygame.time.get_ticks()

    def render_arena(self, screen):
        elapsed = pygame.time.get_ticks() - self.state_started
        left_x, right_x, icon_y = self.width // 4, 3 * self.width // 4, 185
        base_size = 100

        you = self.font_hud.render("YOU", True, (100, 180, 255))
        cpu = self.font_hud.render("CPU", True, (255, 120, 120))
        screen.blit(you, (left_x - you.get_width() // 2, 100))
        screen.blit(cpu, (right_x - cpu.get_width() // 2, 100))

        vs = self.font_arena.render("VS", True, (120, 128, 145))
        screen.blit(vs, (self.width // 2 - vs.get_width() // 2, icon_y - vs.get_height() // 2))

        if self.state == STATE_COUNTDOWN:
            # Both fists bob in sync once per beat before the reveal
            beat_phase = (elapsed % self.countdown_beat) / self.countdown_beat
            offset = int(-abs(math.sin(beat_phase * math.pi)) * 22)
            draw_gesture(screen, "ROCK", (left_x, icon_y + offset), base_size)
            draw_gesture(screen, "ROCK", (right_x, icon_y + offset), base_size)
        elif self.player_choice and self.cpu_choice:
            # Pop the revealed gestures in with a short scale-up
            pop = min(1.0, elapsed / self.reveal_pop) if self.state == STATE_RESULT else 1.0
            size = int(base_
... [3219 more characters omitted]
```

Result

```
The file <scratch>/59_rock_paper_scissor/game/game_engine.py has been updated successfully. (file state is current in
  your context — no need to Read it back)
```

Tool call - Bash: Resize window and unit-check outcome rules and AI bias

```
cd <scratch>/59_rock_paper_scissor && sed -i '' 's/^WIDTH, HEIGHT = 560, 360$/WIDTH, HEIGHT = 640, 400/' main.py &&
  grep -n "WIDTH, HEIGHT =" main.py && ../venv/bin/python - <<'EOF'
import os, sys
os.environ["SDL_VIDEODRIVER"]="dummy"
sys.path.insert(0, ".")
import pygame; pygame.init(); pygame.display.set_mode((640,400))
from game.game_engine import GameEngine
from collections import Counter
e = GameEngine(640,400)
# Task 1: all 9 combos
for p in e.choices:
    for c in e.choices:
        print(p, "vs", c, "->", e.determine_winner(p,c))
# Task 3: adaptive bias when player spams ROCK
import random; random.seed(1)
for _ in range(6): e.player_history.append("ROCK")
picks = Counter(e.cpu_pick() for _ in range(3000))
print("CPU picks after 6x ROCK:", dict(picks), "| note:", e.ai_note)
e.player_history.clear()
picks = Counter(e.cpu_pick() for _ in range(3000))
print("CPU picks with no history:", dict(picks))
EOF
```

Result

```
4:WIDTH, HEIGHT = 560, 360■
pygame 2.6.1 (SDL 2.28.4, Python 3.9.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
ROCK vs ROCK -> TIE
ROCK vs PAPER -> CPU
ROCK vs SCISSORS -> PLAYER
PAPER vs ROCK -> PLAYER
PAPER vs PAPER -> TIE
PAPER vs SCISSORS -> CPU
SCISSORS vs ROCK -> CPU
SCISSORS vs PAPER -> PLAYER
SCISSORS vs SCISSORS -> TIE
CPU picks after 6x ROCK: {'PAPER': 2499, 'ROCK': 263, 'SCISSORS': 238} | note: CPU has noticed you favour ROCK...
CPU picks with no history: {'SCISSORS': 985, 'PAPER': 1021, 'ROCK': 994}
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Bash: Check line endings and apply resize

```
cd <scratch>/59_rock_paper_scissor && file main.py README.md game/*.py && sed -i '' 's/^WIDTH, HEIGHT = 560, 360/
  WIDTH, HEIGHT = 640, 400/' main.py && grep -n "WIDTH, HEIGHT =" main.py
```

Result

```
main.py:             Python script text executable, ASCII text, with CRLF line terminators
README.md:           Unicode text, UTF-8 text
game/button.py:      Python script text executable, ASCII text
game/game_engine.py: Python script text executable, ASCII text
game/icons.py:       Python script text executable, ASCII text
4:WIDTH, HEIGHT = 640, 400
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Write <scratch>/record_game.py

```
"""Headless gameplay recorder: runs the real game loop with scripted mouse/keyboard input
and encodes the frames to MP4.
```

```
usage: record_game.py <game_dir> <out.mp4> <script> [seed] [snapshot_dir]
script: comma list of  t_ms:CHOICE  (click that button) or  t_ms:key_r
"""
import os
import sys

os.environ["SDL_VIDEODRIVER"] = "dummy"
os.environ["SDL_AUDIODRIVER"] = "dummy"

game_dir, out_path, script = sys.argv[1], sys.argv[2], sys.argv[3]
seed = int(sys.argv[4]) if len(sys.argv) > 4 else 0
snap_dir = sys.argv[5] if len(sys.argv) > 5 else None
sys.path.insert(0, game_dir)

import random
import imageio.v2 as imageio
import numpy as np
import pygame

import main as game_main
from game.game_engine import GameEngine

random.seed(seed)
FPS = 30
DURATION_MS = 10_500

pygame.init()
W, H = game_main.WIDTH, game_main.HEIGHT
screen = pygame.display.set_mode((W, H))
engine = GameEngine(W, H)
buttons = {b.choice_name: b.rect.center for b in engine.buttons}

actions = []
for item in script.split(","):
    t, what = item.split(":")
    actions.append((int(t), what))
actions.sort()

# Cursor waypoints: glide to each target 450ms before its click
waypoints = [(0, (W // 2, H // 2))]
for t, what in actions:
    if what in buttons:
        waypoints.append((max(0, t - 450), waypoints[-1][1]))
        waypoints.append((t, buttons[what]))

def cursor_at(t):
    prev = waypoints[0]
    for wp in waypoints:
        if wp[0] >= t:
            (t0, p0), (t1, p1) = prev, wp
            k = 0 if t1 == t0 else (t - t0) / (t1 - t0)
            return (int(p0[0] + (p1[0] - p0[0]) * k), int(p0[1] + (p1[1] - p0[1]) * k))
        prev = wp
    return waypoints[-1][1]

sim_pos = [(W // 2, H // 2)]
pygame.mouse.get_pos = lambda: sim_pos[0]

writer = imageio.get_writer(out_path, fps=FPS, codec="libx264", quality=8, macro_block_size=1,
                            ffmpeg_params=["-pix_fmt", "yuv420p"])
clock = pygame.time.Clock()
start = pygame.time.get_ticks()
pending = list(actions)
last_click = -10_000
frame_no = 0
snap_every = FPS // 2

while True:
    t = pygame.time.get_ticks() - start
    if t > DURATION_MS:
        break
    sim_pos[0] = cursor_at(t)

    while pending and pending[0][0] <= t:
        _, what = pending.pop(0)
        if what == "key_r":
            engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_r))
        else:
            engine.handle_event(pygame.event.Event(pygame.MOUSEBUTTONDOWN, button=1, pos=buttons[what]))
            last_click = t

    engine.update()
    engine.render(screen)

    # Draw a visible cursor (with click ripple) onto a copy of the frame
    frame = screen.copy()
    cx, cy = sim_pos[0]
    if t - last_click < 300:
        pygame.draw.circle(frame, (255, 255, 255), (cx, cy), 8 + (t - last_click) // 15, width=2)
    arrow = [(cx, cy), (cx, cy + 18), (cx + 5, cy + 13), (cx + 9, cy + 21), (cx + 12, cy + 19), (cx + 8, cy + 12), (
  cx + 14, cy + 12)]
    pygame.draw.polygon(frame, (255, 255, 255), arrow)
    pygame.draw.polygon(frame, (0, 0, 0), arrow, width=1)
    writer.append_data(np.transpose(pygame.surfarray.array3d(frame), (1, 0, 2)))
```

```
    if snap_dir and frame_no % snap_every == 0:
        pygame.image.save(frame, os.path.join(snap_dir, f"f{frame_no:04d}_{t:05d}ms.png"))
    frame_no += 1
    clock.tick(FPS)

writer.close()
print(f"wrote {frame_no} frames ({frame_no / FPS:.1f}s) -> {out_path}; final score {engine.player_score}-{engine.
  cpu_score}")
```

Result

```
File created successfully at: <scratch>/record_game.py (file state is current in your context — no need to Read it
  back)
```

Claude · 2026-10-02 17:19 UTC

Before recording, I'll pick a random seed so the "before" video clearly shows wins being scored as losses.

Tool call - Bash: Find seed showing the bug in several rounds

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  python3 -c "
import random
ch=['ROCK','PAPER','SCISSORS']; beats={'ROCK':'SCISSORS','SCISSORS':'PAPER','PAPER':'ROCK'}
seq=['ROCK','PAPER','SCISSORS','ROCK','PAPER']
for s in range(500):
    random.seed(s); c=[random.choice(ch) for _ in seq]
    wins=sum(beats[p]==x for p,x in zip(seq,c)); losses=sum(beats[x]==p for p,x in zip(seq,c))
    if wins>=3 and losses>=1: print(s,c,wins,losses); break
"
```

Result

```
6 ['SCISSORS', 'ROCK', 'PAPER', 'PAPER', 'ROCK'] 4 1
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Bash: Record 10s before-changes video of original game

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  mkdir -p out snaps_before && ./venv/bin/python record_game.py rps_original out/before_changes.mp4 "800:ROCK,2900:
  PAPER,5000:SCISSORS,7100:ROCK,9200:PAPER" 6 snaps_before 2>&1 | grep -v -e "^pygame " -e "Hello from" && ls
  snaps_before | head -30
```

Result

```
[vost#0:0/libx264 @ 0x14e61c3c0] Multiple -pix_fmt options specified for stream 0, only the last option '-pix_fmt
  yuv420p' will be used.
wrote 288 frames (9.6s) -> out/before_changes.mp4; final score 1-4
f0000_00000ms.png
f0015_00547ms.png
f0030_01085ms.png
f0045_01635ms.png
f0060_02175ms.png
f0075_02737ms.png
f0090_03296ms.png
f0105_03837ms.png
f0120_04385ms.png
f0135_04935ms.png
f0150_05487ms.png
f0165_06033ms.png
f0180_06587ms.png
f0195_07139ms.png
f0210_07694ms.png
f0225_08228ms.png
f0240_08760ms.png
f0255_09306ms.png
f0270_09851ms.png
f0285_10411ms.png
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:20 UTC

The recorder only managed about 27 fps in real time, so the video plays slightly fast at 9.6 s. I'll switch it to a virtual clock (exactly 1/30 s per frame) so the timing is exact and repeatable.

Tool call - Bash: Switch recorder to virtual clock and re-record before video

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  python3 - <<'EOF'
p="record_game.py"; s=open(p).read()
s=s.replace('''sim_pos = [(W // 2, H // 2)]''','''sim_pos = [(W // 2, H // 2)]
# Virtual clock: each frame advances exactly 1000/FPS ms, so the video plays at true speed
virtual_ms = [0]
pygame.time.get_ticks = lambda: int(virtual_ms[0])''')
s=s.replace('''writer = imageio.get_writer(out_path, fps=FPS, codec="libx264", quality=8, macro_block_size=1,
                           ffmpeg_params=["-pix_fmt", "yuv420p"])
clock = pygame.time.Clock()
start = pygame.time.get_ticks()''','''writer = imageio.get_writer(out_path, fps=FPS, codec="libx264", quality=8,
  macro_block_size=1)
start = 0''')
s=s.replace('''    frame_no += 1
    clock.tick(FPS)''','''    frame_no += 1
    virtual_ms[0] += 1000 / FPS''')
open(p,"w").write(s)
EOF
grep -n "virtual_ms\|Clock\|start =" record_game.py; rm -rf snaps_before && mkdir snaps_before && ./venv/bin/python
  record_game.py rps_original out/before_changes.mp4 "800:ROCK,2900:PAPER,5000:SCISSORS,7100:ROCK,9200:PAPER" 6
  snaps_before 2>&1 | grep -v -e "^pygame " -e "Hello from"
```

Result

```
61:virtual_ms = [0]
62:pygame.time.get_ticks = lambda: int(virtual_ms[0])
66:start = 0
102:    virtual_ms[0] += 1000 / FPS
wrote 316 frames (10.5s) -> out/before_changes.mp4; final score 1-4
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:20 UTC

The before video is now 10.5 s, ending 1–4 even though the player threw the winning hand in four of the five rounds. I'll check a frame to confirm the bug is visible.

Tool call - Bash: List snapshot names after first click

```
ls <scratch>/snaps_before | sed -n '3,4p'
```

Result

```
f0030_01000ms.png
f0045_01499ms.png
```

Tool call - Read <scratch>/snaps_before/f0030_01000ms.png

```
{
 "file_path": "<scratch>/snaps_before/f0030_01000ms.png"
}
```

Result

```
[image]
```

Tool call - Bash: Set first-to-3 and search seed for after-video storyline

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  sed -i '' 's/def __init__(self, width, height, target_wins=5):/def __init__(self, width, height, target_wins=3):/'
  59_rock_paper_scissor/game/game_engine.py && grep -n "target_wins=3" 59_rock_paper_scissor/game/game_engine.py &&
  SDL_VIDEODRIVER=dummy ./venv/bin/python - <<'EOF' 2>&1 | grep -v -e "^pygame " -e "Hello from"
import sys, random; sys.path.insert(0,"59_rock_paper_scissor")
import pygame; pygame.init(); pygame.display.set_mode((640,400))
from game.game_engine import GameEngine
seq=["ROCK","ROCK","ROCK","ROCK"]
for s in range(2000):
    random.seed(s); e=GameEngine(640,400); res=[]
    for p in seq:
        c=e.cpu_pick(); e.player_history.append(p); res.append((p,c,e.determine_winner(p,c)))
    outs=[r[2] for r in res]
    if outs[0]=="PLAYER" and outs.count("CPU")==3 and outs[3]=="CPU" and res[3][1]=="PAPER":
        print(s,res); break
EOF
```

Result

```
21:    def __init__(self, width, height, target_wins=3):
0 [('ROCK', 'SCISSORS', 'PLAYER'), ('ROCK', 'PAPER', 'CPU'), ('ROCK', 'PAPER', 'CPU'), ('ROCK', 'PAPER', 'CPU')]
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:20 UTC

Seed 0 works. Recording the "after" video:

Tool call - Bash: Record 10s after-changes video of fixed game

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  rm -rf snaps_after && mkdir snaps_after && ./venv/bin/python record_game.py 59_rock_paper_scissor out/after_changes.
  mp4 "400:ROCK,2100:ROCK,3800:ROCK,5500:ROCK,9700:key_r" 0 snaps_after 2>&1 | grep -v -e "^pygame " -e "Hello from";
  ls snaps_after | tr '\n' ' '
```

Result

```
wrote 316 frames (10.5s) -> out/after_changes.mp4; final score 0-0
f0000_00000ms.png f0015_00499ms.png f0030_01000ms.png f0045_01499ms.png f0060_01999ms.png f0075_02500ms.png
  f0090_03000ms.png f0105_03500ms.png f0120_04000ms.png f0135_04500ms.png f0150_04999ms.png f0165_05499ms.png
  f0180_05999ms.png f0195_06499ms.png f0210_06999ms.png f0225_07499ms.png f0240_07999ms.png f0255_08499ms.png
  f0270_08999ms.png f0285_09499ms.png f0300_10000ms.png f0315_10500ms.png
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Claude · 2026-10-02 17:20 UTC

The final score reads 0–0 because R reset the match at the end. I'll check frames at five points: countdown, a correct win, the adaptive-AI message, the banner and the reset.

Tool call - Read <scratch>/snaps_after/f0030_01000ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0030_01000ms.png"
}
```

Result

```
[image]
```

Tool call - Read <scratch>/snaps_after/f0060_01999ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0060_01999ms.png"
}
```

Result

```
[image]
```

Tool call - Read <scratch>/snaps_after/f0210_06999ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0210_06999ms.png"
}
```

Result

```
[image]
```

Tool call - Read <scratch>/snaps_after/f0270_08999ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0270_08999ms.png"
}
```

Result

```
[image]
```

Tool call - Read <scratch>/snaps_after/f0300_10000ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0300_10000ms.png"
}
```

Result

```
[image]
```

Claude · 2026-10-02 17:21 UTC

All four features work in the frames. One cosmetic issue: the scissor blades are thin lines, so I'll widen them and re-record.

Tool call - Edit <scratch>/59_rock_paper_scissor/game/icons.py

```
{
 "replace_all": false,
 "file_path": "<scratch>/59_rock_paper_scissor/game/icons.py",
 "old_string": "        tip = (cx + side * r * 0.55, cy - r * 0.95)\n        base_l = (pivot[0] - side * r * 0.12,
  pivot[1] + r * 0.05)\n        base_r = (pivot[0] + side * r * 0.02, pivot[1] - r * 0.12)\n        poly = [base_l,
  tip, base_r]",
 "new_string": "        tip = (cx + side * r * 0.55, cy - r * 0.95)\n        base_l = (pivot[0] - side * r * 0.22,
  pivot[1] + r * 0.08)\n        base_r = (pivot[0] + side * r * 0.08, pivot[1] - r * 0.2)\n        poly = [base_l,
  tip, base_r]"
}
```

Result

```
The file <scratch>/59_rock_paper_scissor/game/icons.py has been updated successfully. (file state is current in your
  context — no need to Read it back)
```

Tool call - Bash: Re-record after video with wider scissor blades

```
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  rm -rf snaps_after && mkdir snaps_after && ./venv/bin/python record_game.py 59_rock_paper_scissor out/after_changes.
  mp4 "400:ROCK,2100:ROCK,3800:ROCK,5500:ROCK,9700:key_r" 0 snaps_after 2>&1 | grep -v -e "^pygame " -e "Hello from"
```

Result

```
wrote 316 frames (10.5s) -> out/after_changes.mp4; final score 0-0
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Tool call - Read <scratch>/snaps_after/f0060_01999ms.png

```
{
 "file_path": "<scratch>/snaps_after/f0060_01999ms.png"
}
```

Result

```
[image]
```

Claude · 2026-10-02 17:21 UTC

The blades are clearer now. Next I'm updating the game's README: its Controls line was copied from another game ("colored pads"), and it needs a summary of what changed.

Tool call - Bash: Update game README with controls fix and change log

```
cd <scratch>/59_rock_paper_scissor && python3 - <<'EOF'
p="README.md"; s=open(p).read()
s=s.replace("**Controls:** Left-click colored pads to repeat the sequence. Press R to restart after Game Over.",
"**Controls:** Left-click ROCK, PAPER or SCISSORS to throw. The match is first to 3 wins; press R to play again after
  the match ends.")
s=s.replace("""■■■ game/
■   ■■■ button.py
■   ■■■ game_engine.py""","""■■■ game/
■   ■■■ button.py
■   ■■■ game_engine.py
■   ■■■ icons.py""")
s=s.replace("- [] A 10-second video","- [x] A 10-second video").replace("- [] The Chat/LLM","- [x] The Chat/LLM")
s += """

---

## Changes Made (Lab 4 submission - PES1UG25CS806)

| Task | What changed | Where |
|------|--------------|-------|
| 1. Inverted outcome bug | The rule table awarded every winning hand to the CPU. Replaced it with a single `BEATS`
  map (Rock>Scissors, Scissors>Paper, Paper>Rock) so the winner is derived from one source of truth. All 9
  combinations verified. | `game/game_engine.py` `determine_winner` |
| 2. First to X wins | Added `target_wins` (default 3), a `MATCH_OVER` state, a victory banner with the final score,
  input lock while the match is over, and **R** to restart. | `game/game_engine.py` `resolve_round`,
  `render_match_over`, `reset_match` |
| 3. Adaptive AI | The CPU keeps the player's last 6 throws and weights its pick toward the counter of each recent
  move (`1 + 1.5 * count`). It decides *before* seeing the current throw. When one move appears 3+ times a hint shows:
   "CPU has noticed you favour ROCK...". With 6 ROCKs in history the CPU picks PAPER ~83% of the time; with no
  history it is uniform. | `game/game_engine.py` `cpu_pick` |
| 4. Gesture icons + reveal | New `game/icons.py` draws Rock (lumpy fist/stone), Paper (ruled sheet with folded
```

```
  corner) and Scissors (blades + rings) with pygame primitives. Each round plays a synced "ROCK... PAPER... SCISSORS..
  ." countdown where both fists bob, then the real gestures pop in with a scale-up. Buttons show small icons too. |
  `game/icons.py`, `game/button.py`, `game/game_engine.py` `render_arena` |

The window was enlarged to 640x400 (`main.py`) to fit the icon arena.
"""
open(p,"w").write(s)
EOF
grep -n "Controls\|\[x\]\|icons.py" README.md
```

Result

```
37:**Controls:** Left-click ROCK, PAPER or SCISSORS to throw. The match is first to 3 wins; press R to play again
  after the match ends.
76:■   ■■■ icons.py
85:- [x] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
86:- [x] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
87:- [x] The Chat/LLM used page link, with the complete chat history
99:| 4. Gesture icons + reveal | New `game/icons.py` draws Rock (lumpy fist/stone), Paper (ruled sheet with folded
  corner) and Scissors (blades + rings) with pygame primitives. Each round plays a synced "ROCK... PAPER... SCISSORS..
  ." countdown where both fists bob, then the real gestures pop in with a scale-up. Buttons show small icons too. |
  `game/icons.py`, `game/button.py`, `game/game_engine.py` `render_arena` |
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```

Student · 2026-10-13 10:12 UTC

lab 4 with https://github.com/SETAPESU26/59_rock_paper_scissor this repo link for me given in the pdf

Claude · 2026-10-13 10:12 UTC

I'll check the current Lab 4 folder against the handout. Labs 1–3 were redone, but Lab 4 was built at the very start, so first I'll confirm the assigned repo hasn't changed upstream and that the code still runs.

Tool call - Bash: Verify upstream repo, re-test game, check video lengths

```
git ls-remote https://github.com/SETAPESU26/59_rock_paper_scissor HEAD | cut -c1-7; cd "<SE Lab>/SE-Labs/Lab-4" &&
  find . -type f | sort && cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-
  9b38b4e9ffb6/scratchpad && ./venv/bin/pip install -q pygame imageio imageio-ffmpeg 2>&1 | tail -1; cd "<SE Lab>/SE-
  Labs/Lab-4/rock_paper_scissor" && SDL_VIDEODRIVER=dummy <scratch>/venv/bin/python -B - <<'EOF' 2>&1 | grep -v -e
  "^pygame " -e "Hello from"
import pygame, main, random
from collections import Counter
from game.game_engine import GameEngine
pygame.init(); s = pygame.display.set_mode((main.WIDTH, main.HEIGHT)); e = GameEngine(main.WIDTH, main.HEIGHT)
bad = [(p, c) for p in e.choices for c in e.choices
       if e.determine_winner(p, c) != ("TIE" if p == c else ("PLAYER" if {"ROCK":"SCISSORS","SCISSORS":"PAPER",
  "PAPER":"ROCK"}[p] == c else "CPU"))]
print("Task1 rule errors:", bad)
random.seed(1); e.player_history.extend(["ROCK"] * 6)
print("Task3 CPU vs 6x ROCK:", dict(Counter(e.cpu_pick() for _ in range(3000))))
e.player_score = 2; e.player_choice, e.cpu_choice = "ROCK", "SCISSORS"; e.resolve_round()
print("Task2 match winner at 3:", e.match_winner, "| target", e.target_wins)
for _ in range(10): e.update(); e.render(s)
print("render ok")
EOF
cd /private/tmp/claude-501/-Users-aryan-upadhyayyy-Desktop-SE-Lab-/0d633572-0b88-4a26-90c3-9b38b4e9ffb6/scratchpad &&
  for v in before_changes after_changes; do ./venv/bin/python -c "
import imageio.v2 as iio, sys; r = iio.get_reader(sys.argv[1]); m = r.get_meta_data(); print(sys.argv[1].split('/')[-
  1], round(m['duration'], 1), 's', m['size'], m['fps'], 'fps')" "<SE Lab>/SE-Labs/Lab-4/videos/$v.mp4"; done
```

Result

```
72e89d3
./Lab4_Chat_History.docx
./Lab4_Chat_History.pdf
./README.md
./rock_paper_scissor/README.md
./rock_paper_scissor/game/button.py
./rock_paper_scissor/game/game_engine.py
./rock_paper_scissor/game/icons.py
./rock_paper_scissor/main.py
./videos/after_changes.mp4
./videos/before_changes.mp4
Task1 rule errors: []
Task3 CPU vs 6x ROCK: {'PAPER': 2499, 'ROCK': 263, 'SCISSORS': 238}
Task2 match winner at 3: PLAYER | target 3
render ok
before_changes.mp4 10.5 s (560, 360) 30.0 fps
after_changes.mp4 10.5 s (640, 400) 30.0 fps
Shell cwd was reset to /Users/aryan.upadhyayyy/Desktop/SE Lab
```